moovefy

PROVA TÉCNICA

DESENVOLVEDOR PLENO/SÊNIOR

Avaliação Técnica - Desenvolvedor Pleno/Sênior

Prova Remota

Sistema de Gestão de Tarefas com Notificações

Contexto

Desenvolva uma aplicação web para gerenciamento de tarefas de equipes, com sistema de notificações em tempo real.

Requisitos Funcionais

Backend (.NET Core)

1. API RESTful

Implemente os seguintes endpoints:

- **Projetos:**

Método HTTP	Endpoint	Descrição
GET	/api/projetos	Listar todos os projetos;
GET	/api/projetos/{id}	Obter projeto por ID;
POST	/api/projetos	Criar novo projeto;
PUT	/api/projetos/{id}	Atualizar projeto;
DELETE	/api/projetos/{id}	Excluir projeto.

- **Tarefas:**

Método HTTP	Endpoint	Descrição
GET	/api/tarefas	Listar tarefas com filtros: projeto, status, responsável, prazo;
GET	/api/tarefas/{id}	Obter tarefa por ID;
POST	/api/tarefas	Criar nova tarefa;
PUT	/api/tarefas/{id}	Atualizar tarefa;
PATCH	/api/tarefas/{id}/status	Alterar apenas o status da tarefa;
DELETE	/api/tarefas/{id}	Excluir tarefa.

- **Dashboard:**

Método HTTP	Endpoint	Descrição
GET	/api/dashboard/metricas	Obter métricas gerais relacionadas às tarefas.

  - Total de tarefas por status;
  - Tarefas atrasadas;
  - Tarefas concluídas no prazo;
  - Taxa de conclusão.

2. Modelo de Dados Sugerido

Projeto:

Campo	Tipo (sugestão)	Descrição
Id	int	Identificador único do projeto;
Nome	string	Nome do projeto;
Descrição	string	Detalhes sobre o projeto;
DataCriacao	datetime	Data de criação do projeto.

Tarefa:

Campo	Tipo (sugestão)	Descrição
Id	int	Identificador único da tarefa;
Título	string	Título da tarefa;
Descrição	string	Detalhes da tarefa.
Status	enum	Estado atual: Pendente, Em Andamento, Concluída, Cancelada;
Prioridade	enum	Nível de prioridade: Baixa, Média, Alta, Urgente;
ProjetoId	int	Referência ao projeto relacionado;
ResponsavelId	int ou string	ID ou nome do responsável pela tarefa;
DataCriacao	datetime	Data de criação da tarefa;

DataPrazo	datetime	Prazo final para conclusão;
DataConclusao	datetime	Data em que a tarefa foi concluída.

3. Regras de Negócio:

- Tarefa não pode ser excluída se estiver com status "Em Andamento";
- Ao atribuir/reatribuir tarefa, enviar notificação ao responsável;
- Tarefas com prazo vencido devem ser sinalizadas automaticamente;
- Um projeto só pode ser excluído se não possuir tarefas vinculadas;
- Status de tarefa só pode seguir o fluxo: Pendente → Em Andamento → Concluída;
- Ao concluir tarefa, definir automaticamente a DataConclusao.

Frontend

Telas Necessárias

1. **Lista de Projetos:**
    - Visualização em cards ou tabela;
    - Opções de criar, editar, excluir.
2. **Lista de Tarefas:**
    - Filtros (por projeto, status, responsável, prazo);
    - Ordenação;
    - Indicador visual de tarefas atrasadas;
    - Opções de criar, editar, excluir, alterar status.
3. **Formulários:**
    - Criar/Editar Projeto;
    - Criar/Editar Tarefa.
4. **Dashboard:**
    - Gráficos/cards com métricas;
    - Visão geral do andamento.

Requisitos de UX:

- Interface responsiva (mobile-friendly);
- Feedback visual para ações (loading, sucesso, erro);

- Validação de formulários;
- Confirmação antes de excluir.

Requisitos Técnicos

Obrigatórios

Backend

- .NET Core 6.0 ou superior;
- Banco de dados relacional (SQL Server, PostgreSQL ou MySQL);
- Arquitetura em camadas ou Clean Architecture;
- Pelo menos **um** design pattern aplicado (Repository, Unit of Work, Strategy, Factory, etc.).
- ORM (Entity Framework Core ou Dapper);
- Documentação da API (Swagger/OpenAPI);
- Tratamento de exceções e validações;
- Logging básico.

Frontend:

- Angular, React ou Vue.js;
- Consumo da API REST;
- Gerenciamento de estado (Context API, Redux, Vuex, etc.);
- Componentização adequada.

DevOps:

- Docker Compose para subir toda a aplicação (backend + banco + frontend);
- README com instruções claras de execução;
- Migrations ou scripts de criação do banco.

Diferenciais (Especialmente para Sênior):

- **Testes Unitários** (mínimo 50% cobertura nas regras de negócio);
- **Autenticação/Autorização** (JWT);
- **SignalR ou WebSockets** para notificações em tempo real;
- **CI/CD** (GitHub Actions, GitLab CI ou similar);
- **Logs estruturados** (Serilog ou NLog);

- **Cache** em endpoints relevantes (Redis ou In-Memory);
- **Testes de Integração;**
- **Kubernetes manifests** (deployment, service, ingress básicos);
- **Rate Limiting** na API;
- **Documentação técnica** da arquitetura.

Critérios de Avaliação

Critério	Peso	Detalhes
Funcionalidade	25%	Requisitos implementados corretamente, aplicação funciona end-to-end
Qualidade do Código	25%	Clean code, SOLID, nomenclatura, organização, patterns
Arquitetura	20%	Separação de responsabilidades, escalabilidade, manutenibilidade
Documentação	15%	README claro, decisões técnicas documentadas, API docs
Testes	10%	Cobertura e qualidade dos testes unitários/integração
DevOps	5%	Docker, facilidade de executar, CI/CD

O que buscamos em cada nível?

Pleno:

- Implementa todos os requisitos obrigatórios;
- Código organizado e legível;
- Entende os conceitos aplicados;
- Documentação básica funcional.

Sênior:

- Todos os itens do exigidos no Pleno;
- Implementa vários diferenciais;
- Código com alta qualidade (SOLID, patterns bem aplicados);
- Testes significativos e bem estruturados;
- Preocupação com escalabilidade e performance;
- Documentação técnica completa;
- Demonstra visão arquitetural.

Entrega

Repositório Git:

Envie o link do repositório (GitHub, GitLab ou Bitbucket) para: **brahiam.leme@moovefy.com.br;**

O repositório deve ser **público**.

README.md deve conter:

1. **Descrição do Projeto:**
    - Breve resumo do que foi desenvolvido.
2. **Tecnologias Utilizadas:**
    - Stack completo (backend, frontend, banco, libs principais).
3. **Como Executar:**
    - Pré-requisitos;
    - Comandos para rodar com Docker;
    - Comandos alternativos (caso alguém queira rodar sem Docker);
    - Credenciais padrão (se houver autenticação);
    - Porta da API e do frontend.
4. **Decisões Técnicas:**
    - Por que escolheu determinada arquitetura?
    - Quais patterns aplicou e por quê?
    - Trade-offs que considerou.
5. **Uso de IA no Desenvolvimento:**
    - **Seja transparente!** Como usou ferramentas de IA?
    - Para que usou? (boilerplate, pesquisas, revisão, testes?)
    - O que adaptou ou melhorou do código gerado?
6. **Melhorias Futuras:**
    - O que faria diferente com mais tempo?
    - Funcionalidades que gostaria de adicionar.
7. **Testes:**
    - Como rodar os testes;

- Cobertura alcançada.

Extras (Opcional, mas valorizado):

- **Vídeo de demonstração** (3-5 minutos):
  - Mostrando a aplicação funcionando;
  - Explicando decisões técnicas principais;
  - Pode ser no Loom, YouTube (não listado) ou similar.
- **Screenshots** da aplicação rodando;
- **Diagrama de arquitetura** (pode ser simples, feito em draw.io, Excalidraw, etc.).

Sobre o Uso de Inteligência Artificial

Incentivamos o uso de IA!

Ferramentas como GitHub Copilot, ChatGPT, Claude, etc., são parte do dia a dia de desenvolvimento moderno.

O que avaliaremos sobre seu uso de IA:

- **Como você usa**: Para boilerplate? Pesquisas? Revisão de código? Testes?
- **Seu senso crítico**: Você valida e entende o código gerado ou cópia sem questionar?
- **Sua capacidade de adaptação**: Você consegue melhorar e adaptar o código da IA para o contexto?
- **Seu conhecimento**: Você consegue explicar e defender as escolhas técnicas?

O que NÃO queremos:

- Copiar projetos prontos inteiros sem entender;
- Código gerado por IA cheio de bugs e não testado;
- Não conseguir explicar o que foi implementado.

Lembre-se: Um desenvolvedor que usa IA de forma eficiente é mais produtivo. Mas um desenvolvedor que não entende o código que a IA gera é um risco para o time.

Dúvidas?

Se tiver qualquer dúvida sobre os requisitos, pode entrar em contato:

- **Email:** rh@moovefy.com.br ou através do whatsapp;
- As dúvidas serão encaminhadas para o gestor técnico e respondidas o mais rápido possível.

Prédio 01
R. dos Rádio-Amadores, 4-35
Jardim Brasil, Bauru - SP
CEP: 17 011-090

Prédio 02
R. Vinte e Nove de Outubro, 7-50
Vila Brunhari, Bauru - SP
CEP: 17 011-050

www.moovefy.com
(14) 3206-0321

Dicas Finais

Foque no essencial primeiro - Melhor entregar todos os requisitos obrigatórios funcionando do que tentar fazer tudo e não terminar.

Commits organizados - Mostre seu processo de pensamento através de commits bem descritos.

Código limpo > Código complexo - Preferimos código simples e bem escrito a código "inteligente", mas difícil de entender.

Documente decisões importantes - Especialmente trade-offs que teve que fazer.

Teste antes de enviar - Garanta que outra pessoa consegue rodar seu projeto seguindo o README.

Boa sorte! Estamos ansiosos para ver sua solução!